KNAPSACK PROBLEMS AND DYNAMIC PROGRAMMING

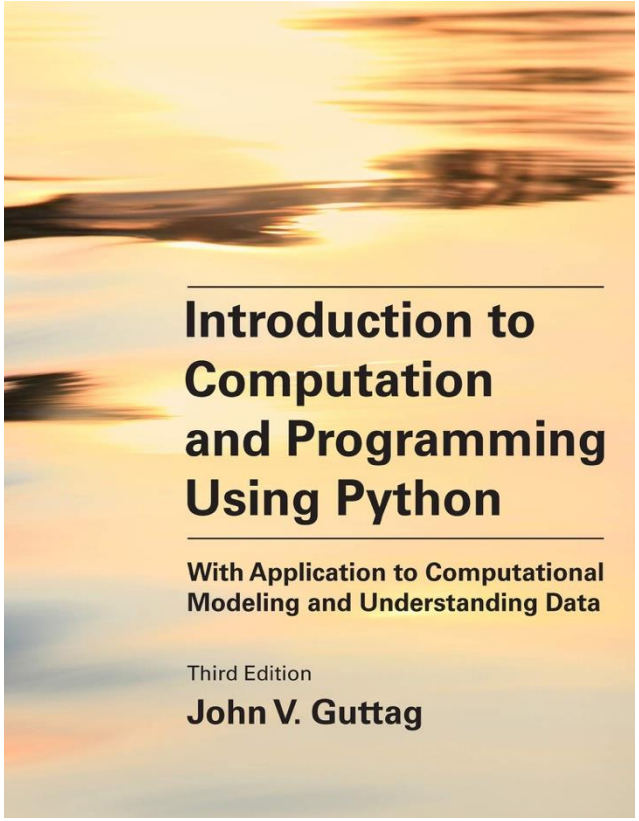
(download slides and .py files from website to follow along)

John Guttag

MIT Department of Electrical Engineering and Computer Science

Relevant Reading

- Today
  - Chapter 15
- Next lecture
  - 17.1 – 17.3
  - 18.1 – 18.3



All code and an errata sheet at
https://github.com/guttag/Intro-to-Computation-and-Programming

Reminders

- Optional recitation on Wednesday, Feb. 11, at 11, here
- Microquiz 1 on **Tuesday**, Feb. 17, at 11, here
  - 35 minutes
  - Bring charged computers
- Next lecture on Wednesday, February 18
  - First problem set also due that day

Previous Lecture

- **Knapsack problems**
  - Find a subset with the maximum total value, subject to a capacity limit (upper bound) on their total weight
  - Two approaches to **exhaustive enumeration.**
    - Take exponential time
  - **Greedy** solution
    - Order items by priority, then add in order items that meet constraint
    - Fast (log linear), but not guaranteed to find an optimal solution
- **Introduction to Dynamic Programming**
- **Why knapsack is a suitable candidate for DP**

Which DP Method?

- Tabular
  - Identify order in which subproblems must be solved
    - Order not always obvious
  - Fill in table mapping problem to result
    - Might create entries that are never used
  - Usually iterative
- Memoization
  - Write a solution (typically recursive)
    - Overhead of recursive calls can be meaningful
  - Cache results of subproblems as they are solved
  - Reuse cached results in future calls
- Memoization or tabular?

Modify decision_tree to Use a Memo

- Try memorization first
- Add **memo** (a dictionary) as a **third argument**
- **Key** of memo is a tuple of ints
  - (remaining items, available capacity)
  - Remaining items represented by `len(items)`
- **Value** associated with key is best solution, plus its optimal total value
- Modifications to body of function
  - First, **check whether** the optimal choice of items given the available cost is **already in the memo**
  - Last, **update the memo**

Why This Key for Our Memo?

- Why can we just use `(len(items), capacity)` as our key? Why don't we need to keep the full list of items left to consider?
- Which items are left for any node at the same level in the tree?
  - They are always the same (as shown on the next slide).
  - So, no need to key on actual items, can just use length, which is different for each level of the tree (because we are using slicing rather than indexing into original list)
    - Would be faster to using indexing

Implementation

```
def knapsack_memo(items, capacity, memo = None):
    global num_calls # for pedagogical reasons
    num_calls += 1
    # Recursively explore tree
    if not items or capacity == 0: # base case
        return (), 0
    if memo == None:
        memo = {}
    if (len(items), capacity) in memo:
        return memo[(len(items), capacity)]
    item = items[0] # Get first item not yet seen
    if item.get_cost() > capacity: # Does current item fit
        return knapsack_memo(items[1:], capacity, memo)
    ...

    # Update memo and return better choice
    if with_value > without_value:
        memo[(len(items), capacity)] = (with_item, with_value)
        return with_item, with_value
    else:
        memo[(len(items), capacity)] = (without_item, without_value)
        return without_item, without_value
```

Memo handling

Testing Code

```
def test_funcs(func, num_items_list, limit, max_val, max_cost, verbose):
    for num_items in num_items_list:
        print(f'Test {func.__name__} for {num_items} items,'
              f' with capacity {limit}')
        names, values, calories = generate_foods(num_items,
                                                 max_val, max_cost)
        foods = build_menu(names, values, calories)
        random.seed()
        random.shuffle(foods)
        solution, value = func(foods, limit)
        print(f'Total value of items taken = {value}')
        if verbose:
            for item in solution:
                print(f'   {item}')
```

```
capacity, max_val, max_cost, num_items = 10, 5, 2, 20
seed = 1
num_calls = 0
random.seed(seed)
test_funcs(decision_tree, (num_items,), capacity, max_val, max_cost, True)
random.seed(seed)
test_funcs(knapsack_memo, (num_items,), capacity, max_val, max_cost, True)
```

Stop and Think

- I set a seed before calling `test_funcs`, and then reset seed during execution of `test_funcs`
- What is being accomplished by this?

Stop and Think

- I set a seed before calling `test_funcs`, and then reset seed during execution of `test_funcs`
- What is being accomplished by this?

Setting seed before call, ensures that data generated by `test_funcs` is same for both methods being tested

Using a random seed before shuffling, tests that value found does not depend upon order of items

Compare Results of Two Methods

Test decision_tree for 20 items, with capacity 10
Total value of items taken = 31
food13: <4, 1>
food14: <4, 2>
food0: <2, 1>
food1: <5, 2>
food3: <3, 1>
food7: <4, 1>
food5: <4, 1>
food15: <5, 1>

Test knapsack_memo for 20 items, with capacity 10
Total value of items taken = 31
food7: <4, 1>
food8: <4, 2>
food19: <2, 1>
food3: <3, 1>
food5: <4, 1>
food1: <5, 2>
food15: <5, 1>
food13: <4, 1>

Your results may vary

Test on Large Sets of Items

```
def knapsack_memo(items, capacity, memo = None):
    """items a list of Item, capacity >= 0
       Solve the 0/1 knapsack problem.
       Return a tuple of an optimal subset of items and their
       total value"""
    global num_calls
    num_calls += 1
```

.

.

.

```
random.seed(1)
for exp in range(1, 11):
    num_calls = 0
    num_items = 2**exp
    test_funcs(knapsack_memo, (num_items,), 1000, 100, 300, verbose = False)
    print(f'Number of invocations of knap_sack_memo = {num_calls:,}')
```

Do We Dare Try This?

len(items)	2**len(items)
2	4
4	16
8	256
16	65,536
32	4,294,967,296
64	18,446,744,073,709,551,616
128	340,282,366,920,938,463,463,374,607,431,768,211,456
256	115,792,089,237,316,195,423,570,985,008,687,907,853,269,984,665,640,564,039,457,584,007,913,129,639,936
512	Really really big number
1024	Absolutely huge

Stop and Think

- Is 2^{1024} larger than

  number of people on Earth

  number of grains of sand on earth

  number of atoms on earth

  number of atoms in universe

How Big Is 2^{1024} ?

- Is 2^{1024} larger than

  number of people on Earth Yes

  10^{299} times larger

  number of grains of sand on earth Yes

  10^{290} times larger

  number of atoms on earth Yes

  10^{258} times larger

  number of atoms in universe Yes

  10^{228} times larger

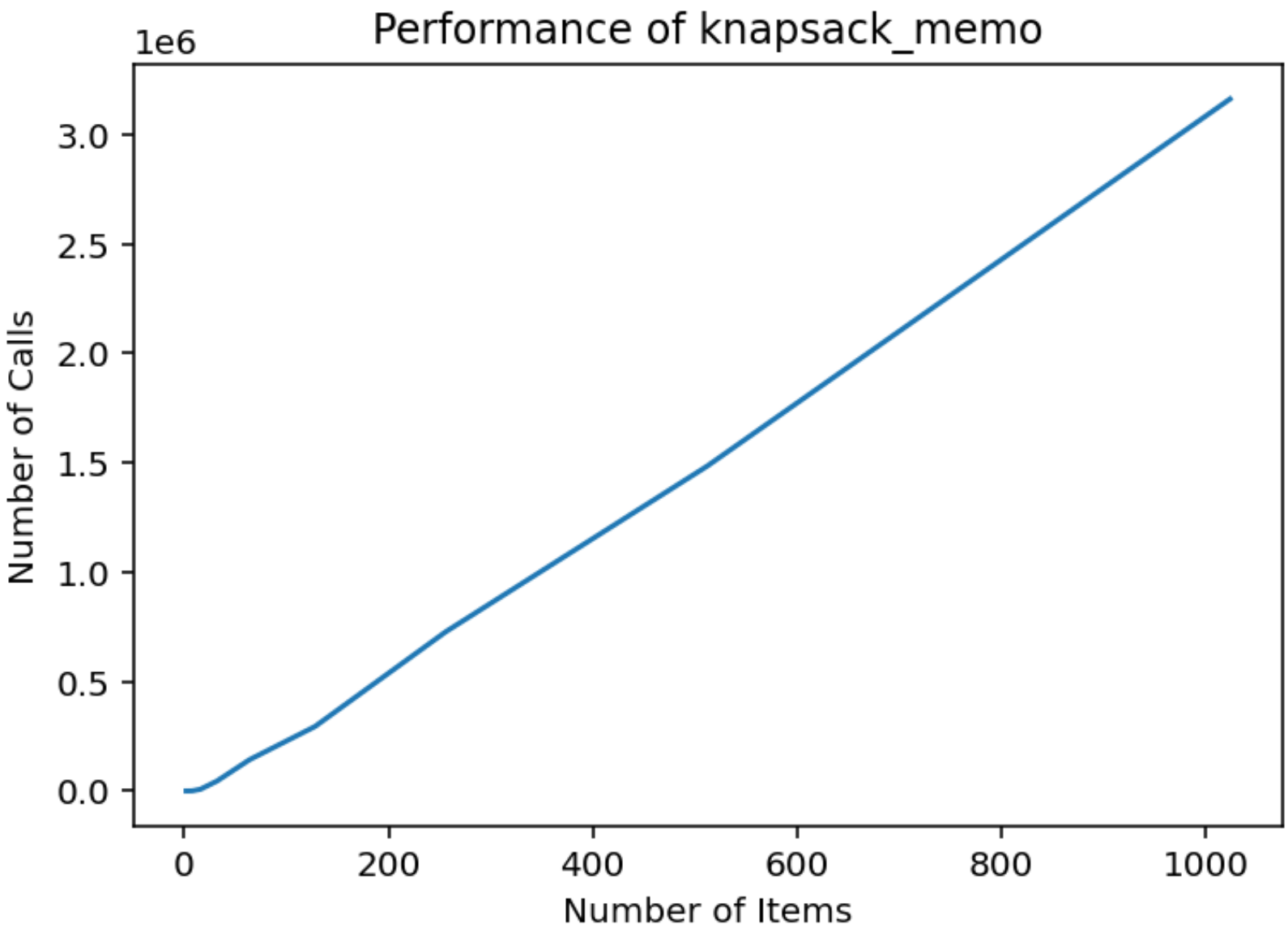
Let’s Dare

Adobe stock image

Performance

len(items)	2**len(items)	Number of calls in DP
2	4	7
4	16	31
8	256	418
16	65,536	9231
32	4,294,967,296	49,059
64	18,446,744,073,709,551,616	141,473
128	340,282,366,920,938,463,463,374,607,431,768,211,456	324,257
256	115,792,089,237,316,195,423,570,985,008,687,907,853,269,984,665,640,564,039,457,584,007,913,129,639,936	882,011
512	Really really big number	1,624,409
1024	Absolutely huge	3,209,582

Visualizing the Trend



Tabular Implementation

```python
def knapsack_tabular(items, capacity):
    n = len(items)

    # Initialize the DP table with all 0s,
    # with (n + 1) rows and (capacity + 1) columns
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Fill the DP table
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if items[i-1].get_cost() <= w:  # If current item can be included
                # Max of including the item or not including the item
                dp[i][w] = max(dp[i-1][w],
                               dp[i-1][w-items[i-1].get_cost()] +\
                                   items[i-1].get_value())
            else:
                dp[i][w] = dp[i-1][w]  # Exclude the item

    # Traceback to find the items in the optimal solution
    result_value = dp[n][capacity]
    w = capacity
    selected_items = []

    for i in range(n, 0, -1):
        # If the current item is included in the optimal solution
        if dp[i][w] != dp[i-1][w]:
            selected_items.append(items[i-1])
            w -= items[i-1].get_cost()  # Reduce the remaining weight

    return selected_items, result_value
```

Performance Comparison

```
for func in (knapsack_memo, knapsack_tabular):
    num_calls = 0
    random.seed(1)
    start_time = time.time()
    test_funcs(func, (1024,), 1000, 100, 300, False)
    end_time = time.time()
    tot_time = end_time - start_time
    print(f'Elapsed time for {func.__name__} = {tot_time:.2f} seconds')
```

Test knapsack_memo for 1024 items, with capacity 1000
Total value of items taken = 4590
Elapsed time for knapsack_memo = 6.41 seconds

Test knapsack_tabular for 1024 items, with capacity 1000
Total value of items taken = 4590
Elapsed time for knapsack_tabular = 0.37 seconds

Why the difference?

Performance Comparison

```
for func in (knapsack_memo, knapsack_tabular):
    num_calls = 0
    random.seed(1)
    start_time = time.time()
    test_funcs(func, (1024,), 1000, 100, 300, False)
    end_time = time.time()
    tot_time = end_time - start_time
    print(f'Elapsed time for {func.__name__} = {tot_time:.2f} seconds')
```

Test knapsack_memo for 1024 items, with capacity 1000
Total value of items taken = 4590
Elapsed time for knapsack_memo = 6.41 seconds

Test knapsack_tabular for 1024 items, with capacity 1000
Total value of items taken = 4590
Elapsed time for knapsack_tabular = 0.37 seconds

Tabular method uses iteration rather than recursion
But is limited by an implicit assumption we will examine later

How Can DP Be So Fast

- Problem is inherently exponential
- Have we overturned the laws of the universe?
- Is dynamic programming a miracle?

- Not in the liturgical sense
- But computational complexity can be subtle

Inputs to First call of knapsack_memo

- items – list of length n
  - Each element is a pair of numbers
- capacity – a number

Complexity

- Runtime depends upon how many recursive calls are made
  - This also determines size of memo
- This is polynomial in **number of distinct pairs (`items`, `capacity`)**
  - Number of distinct sets of items * number of distinct remaining capacities
- Possibilities for **`items`** argument
  - Slices of the original **`items`** input from each index to end
  - Worst case, length of items
- Possibilities for **capacity** argument trickier to analyze

How Many Possibles Values for capacity?

- Possibilities are the value of original `capacity` minus sum of weights for any subset of `items`, while **remaining nonnegative**
- Bounded by the number of **distinct sums of weights that don't exceed** `capacity`
- If weights are discretized (e.g., integral), this is ***linear in the value*** of `capacity`

When Can this Complexity Be a Problem?

- In our large item set testing, we created Items by randomly selecting
  - integers between 1 and 100 for values
  - integers between 1 and 300 for weights
- When the **number of items increases**, so does the **chance that items have the same cost**, making shared subproblems more likely
- When might **combinations** of items be more likely to have **different weights**?
  - Allow non-integer weights

Let's Try Ours on More Distinct Combinations

```
def generate_foods(num_foods, max_val, max_cal):
    names = [f'food{n}' for n in range(num_foods)]
    values = [random.randint(1, max_val) for _ in range(num_foods)]
    calories = [random.randint(1, max_cal)*random.random()\
                for _ in range(num_foods)]
    return names, values, calories
```

Try it

Performance

len(items)	Number of calls Integer weights	Number of calls float weights
2	7	7
4	31	31
8	418	511
16	9231	118,406
32	49,059	?
64	141,473	?
128	324,257	?
256	882,011	?
512	1,624,409	?
1024	3,209,582	?

Dynamic programming doesn't help if there are very few overlapping subproblems

Time to Think

- Look at implementation of tabular method
- Will it work if values are floats?

Time to Think

- Look at implementation of tabular method
- Will it work if values are floats?

```
def knapsack_tabular(items, capacity):
    n = len(items)

    # Initialize the DP table with all 0s,
    # with (n + 1) rows and (capacity + 1) columns
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
```

Size of table depends not on capacity, but on number of possible combinations of weights

And we don't know what this number will be

Some Other Knapsack Problems

- Multidimensional: items have multiple "weights," e.g., weight and volume, and must satisfy multiple constraints
- Multiple-choice: must pick exactly one item from each group
- Interaction of items, e.g.,
  - Some items cannot both be chosen
  - Some items have values that depend upon other items being chosen
- Multiple knapsacks: items distributed among them
- Problem Set 1 has multiple knapsacks and interaction

Multiple Knapsack Problem

- Multiple knapsacks (let number be m)
  - Each with its own capacity, C_k
- Each of n items can go in at most one knapsack
- Choose $x_{i,k} \in \{0, 1\}$ s.t. 1 iff item i is in knapsack k

$$\max \sum_{i=1}^{n} \sum_{k=1}^{m} x_{i,k} * i.\mathrm{val}$$

Maximize total value

- Subject to constraints

No knapsack is overloaded

$$\sum_{i=1}^{n} x_{i,k} * i.\mathrm{cost} \leq C_k \ \forall k$$

$$\sum_{k=1}^{m} x_{i,k} \leq 1 \ \forall i$$

No item in more than one knapsack

Strictly Harder than Normal 0/1 Knapsack Problem

- Which items to take + where to put them
- But if number of knapsacks is small, not too bad
  - As you will see in problem set 1
- Problem set based upon using DP to solve 0/1 knapsack problem
  - Which you already know how to do
- Trick is to think about how to reduce a multiple knapsack problem with interaction to the classic problem
- More about thinking than coding

Reduction to a Solved Problem

Transforming a new problem to a problem with an **existing solution** is an important problem-solving technique

Almost never have to invent a new algorithm

But need to **be familiar with previously studied problems** and their solutions

And the cleverness to imagine the transform

Break

Look at Using DP to Solve a Different Problem

- How does a spelling corrector work?
- Checks to see if target word is in dictionary
- If not in dictionary looks for a word (or set of words) that are similar to the target, and suggests those
- Question 1: How to measure similarity of strings
- Question 2: How to compute that measure quickly

Measuring Similarity of Strings (s1 and s2)

- Edit distance (Levenshtein distance) is the number of simple edits that need to made to s1 to transform it to s2
- Possible edits
  - Insert a single character in s1
  - Delete a single character from s1
  - Substitute one character for another in s1

Stop and Think

- What is the edit distance Between ”out” and
  - tout
  - route
  - grouch

Stop and Think

- What is the edit distance between ”out” and
  - tout – 1 (one insertion)
  - route – 2 (two insertions)
  - grouch – 4 (two insertions, one sub, one deletion)

Looking at It as a Recurrence

Let D(i, j) = distance between s[1..i] and t[1..j]

D(i, 0) = i Delete all chars in i

D(0, j) = j Add all chars in j

D(i, j) = min(D(i-1, j) + 1, Add 1 char
D(I, j-1) + 1, Delete 1 char
D(i-1, j-1) + 1 if s[i] ≠ t[j] else 0)
Sub 1 char unless already the same

Let's Implement It

```
def levenshtein_brute_force(s1, s2):
    # return number of edits needed
    if s1 == '':
        return len(s2) # Insert all remaining s2 characters
    if s2 == '':
        return len(s1) # Delete all remaining s1 characters
    if s1[0] == s2[0]: # No increase in distance
        return levenshtein_brute_force(s1[1:], s2[1:])
    else:
        insert = 1 + levenshtein_brute_force(s1, s2[1:])
        delete = 1 + levenshtein_brute_force(s1[1:], s2)
        replace = 1 + levenshtein_brute_force(s1[1:], s2[1:])
        return min(insert, delete, replace)
```

Let's Test It

```
def test_levenshtein(s1, fcn):
    words = ['out', 'tout', 'route', 'grouch']
    for s2 in words:
        print(f'Distance between {s1} and {s2} is '
              f'{fcn(s1, s2)}')
```

```
test_levenshtein('out', levenshtein_brute_force)
```

L_distance between out and out is 0
L_distance between out and tout is 1
L_distance between out and route is 2
L_distance between out and grouch is 4

Seems fast, but is it fast enough for a spelling corrector?

Use It to Build a Spelling Corrector, Get Data

```
def correct_spelling(alg, max_dist, sentence):
    print(f'Start to correct spelling of "{sentence}"\n')
    dictionary = get_words() # get a list of ~60k allowable words
    print(f'Use a dictionary of {len(dictionary):,} words\n')
    input_words = sentence.split(' ')
```

`alg`: the function used to compute the distance between words
`max_dist`: maximum distance it will return
`sentence`: string of words to check spelling of

Use It to Build a Spelling Corrector, Find Possible Corrections

```
for src in input_words:
    print(f'Check spelling of "{src}"')
    start = time.time()
    min_dist = float('inf')
    is_word = False
    results = []
    for dest in dictionary:
        dist = alg(src, dest)
        if dist == 0:
            is_word = True
            break
        if dist <= min_dist:
            min_dist = dist
            results.append((dest, dist))
```

Use It to Build a Spelling Corrector, Produce Output

```
if is_word:
    print(f'"{src}" is in dictionary')
else:
    if min_dist > max_dist:
        print(f'No words within distance {max_dist} of "{src}"')
    else:
        corrections = [results[i][0] for i in range(len(results))
           if results[i][1] <= min_dist]
        print(f'Corrections of distance {min_dist} for "{src}" are:')
        print(corrections)
end = time.time()
print(f'Time taken = {end - start:.2f} seconds\n')
```

Nested Loop Where the Work Is Done

```
for src in input_words:
    print(f'Check spelling of "{src}"')
    start = time.time()
    min_dist = float('inf')
    is_word = False
    results = []
    for dest in dictionary:
        dist = alg(src, dest)
        if dist == 0:
            is_word = True
            break
        if dist <= min_dist:
            min_dist = dist
            results.append((dest, dist))
```

Complexity of this section

O(len(input_words)*size of dictionary)*time to compute distance

Let's Try It

```
sentence = 'Ths is a tst of spelimg xpgqrz'
correct_spelling(levenshtein_brute_force, 3, sentence)
```

Given About 20 Minutes, It Produces

Corrections of distance 2 for "Ths" are:
['as', 'he', 'hi', 'ho', 'is', 'oh', 'us', 'ads', 'ass', 'bus',
'chi', 'gas', 'has', 'his', 'ids', 'ifs', 'ins', 'its', 'mho',
'ohm', 'oho', 'ors', 'phi', 'pus', 'she', 'shy', 'the',
'thy', 'ups', 'vas', 'was', 'who', 'why', 'yes', 'bahs',
'mhos', 'ohms', 'this', 'thus', 'whys']
Time taken = 7.54 seconds

"is" is in dictionary
Time taken = 0.00 seconds

"a" is in dictionary
Time taken = 0.00 seconds

Corrections of distance 1 for "tst" are:
['tat', 'tit', 'tot', 'test']
Time taken = 6.08 seconds

of" is in dictionary
Time taken = 0.00 seconds

Corrections of distance 2 for "spelimg" are:
['spewing', 'spelling']
Time taken = 801.10 seconds

No words within distance 3 of "xpgqrz"
Time taken = 442.49 seconds

Implementation of Levenshtein is exponential in length of word
~O(5**(len(s1) + len(s2))

Might DP Help?

- Optimal substructure because edit distance is sum of edit distances of substrings. E.g., edit distance of w[0] + edit distance of rest
- Overlapping subproblems because many different ways of editing start of string can lead to same problem for remainder of string

Recall the Recurrence

Let $D(i, j)$ = distance between $s[1..i]$ and $t[1..j]$

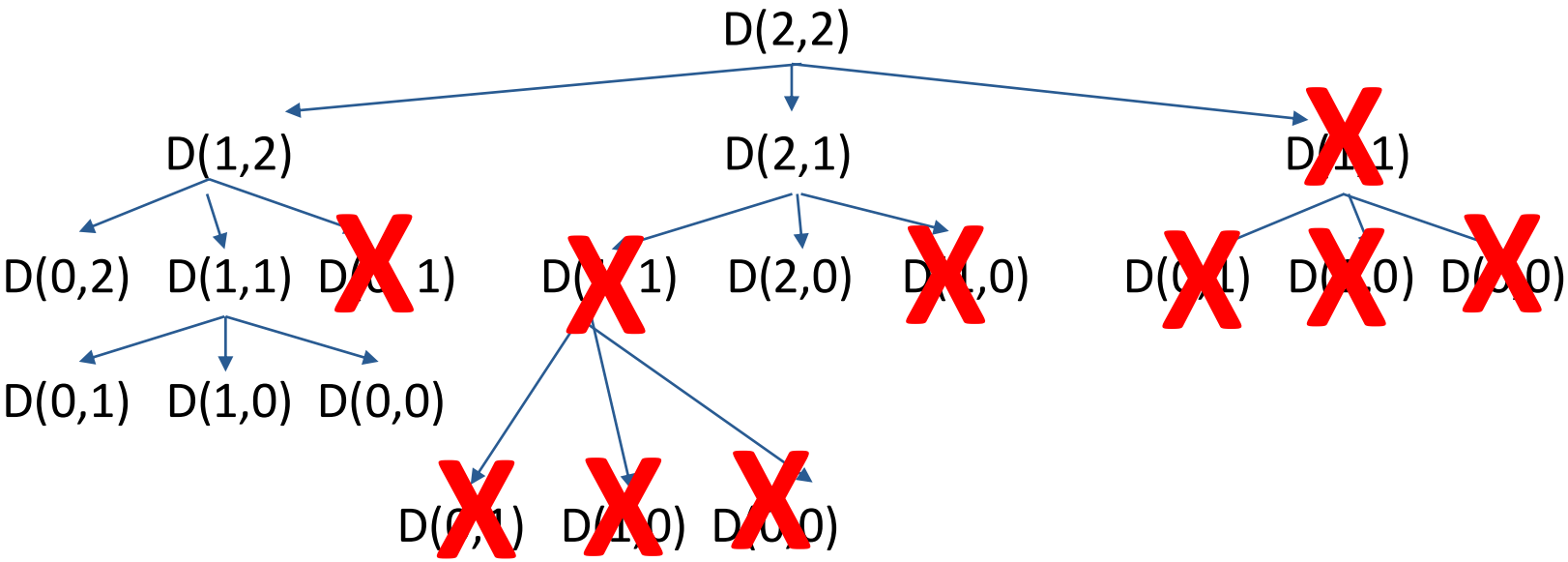
$D(i, 0) = i$

$D(0, j) = j$

$$D(i, j) = \min(D(i-1, j) + 1,$$
$$D(I, j-1) + 1,$$
$$D(i-1, j-1) + 1 \text{ if } s[i] \neq t[j] \text{ else } 0)$$

Like Fibonacci, but worse. Three cases instead of two.

Consider Two Strings of Length Two



Eliminated about the half the nodes here, but the deeper the tree the larger the fraction of nodes eliminated

Using Tabular DP

```
def levenshtein_dp(s1, s2):
    dp = [[0] * (len(s2) + 1) for _ in range(len(s1) + 1)]
    # Initialize base cases
    for i in range(len(s1) + 1):
        dp[i][0] = i
    for j in range(len(s2) + 1):
        dp[0][j] = j
    # Fill DP table
    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            dp[i][j] = min(
                dp[i - 1][j] + 1,       # Deletion
                dp[i][j - 1] + 1,       # Insertion
                dp[i - 1][j - 1] +
                  (0 if s1[i - 1] == s2[j - 1] else 1))  # Substitution
    return dp[len(s1)][len(s2)]
```

Fast Enough?

```
sentence = 'Ths is a tst of spelimg xpgqrz'
correct_spelling(levenshtein_dp, 3, sentence)
```

Produces the same answers in about **3 seconds**

Wrapping Up This Unit

- Many problems of practical importance can be formulated as **optimization problems**
- **Greedy algorithms** often provide adequate (though not necessarily optimal) solutions
- Finding an optimal solution is usually **exponentially hard**

Wrapping Up This Unit, cont.

- **Dynamic programming** often yields good performance for a subclass of optimization problems—those with **optimal substructure** and **overlapping subproblems**
  - Solution **always correct**
  - **Fast** under the right circumstances
- Next unit: stochastics and simulation models

Reminders

- Optional recitation on Wednesday, Feb. 11, at 11, here
- Microquiz 1 on **Tuesday**, Feb. 17, at 11, here
  - 35 minutes
  - Bring charged computers
- Next lecture on Wednesday, February 18
  - First problem set also due that day

Good luck on the quiz next **Tuesday**